NEXUS — Module Log

Module 1: Core Foundation ✅

Files added

```
nexus/__init__.py
nexus/core/__init__.py
nexus/core/config.py
nexus/core/logging.py
nexus/core/di.py
nexus/core/interfaces.py
nexus/core/exceptions.py
tests/__init__.py
tests/test_config.py
tests/test_logging.py
tests/test_di.py
tests/test_interfaces.py
pyproject.toml
requirements.txt
.env.example
.gitignore
Dockerfile
README.md
```

Verified: `python -m unittest discover -s tests -v` → 32/32 passing, zero warnings. Also verified with an end-to-end smoke import (load config → configure logging → resolve from DI container) outside the test suite.

Not verified in this sandbox: `docker build` (Docker is not installed here) and installation of the `dev` extras (no network access in this sandbox). Both follow standard, reviewable patterns and should be validated in your local/CI environment before Module 2 lands on top of them.

Architecture

Module 1 establishes the layer every later module depends on, and nothing else:

- **`config.py`** — Immutable, validated configuration loaded from env vars / `.env`, with clear precedence (kwargs > real env > `.env` file > defaults) and secret redaction for safe logging. Built on `dataclasses` only — no pydantic

dependency
at this layer, so the foundation stays importable anywhere.

- **`logging.py`** — Structured JSON logging to a rotating file (for audit/ingestion) plus human-readable console output (for development), also stdlib-only.
- **`interfaces.py`** — `Protocol` definitions (`LLMProvider`, `SpeechToTextProvider`, `TextToSpeechProvider`, `MemoryStore`, `ActionExecutor`, `GuardrailEngine`, `Agent`) that every concrete implementation in later modules must satisfy. Using structural typing (Protocols) rather than abstract base classes means plugin authors can satisfy an interface without importing NEXUS's class hierarchy.
- **`di.py`** — A small, thread-safe DI container resolving providers by interface type, supporting both singleton and non-singleton (fresh-instance) registration.
- **`exceptions.py`** — One exception hierarchy rooted at `NexusError`, with

`GuardrailViolation` and `PermissionDeniedError` called out explicitly since those are the ones that must never be silently swallowed anywhere in the codebase.

Why this order: Every other module — the LangGraph agents, the voice pipeline, the ADB/Appium/Playwright executors, the guardrail engine itself — needs a place to read config from, a place to log to, a way to be swapped via DI, and a shared vocabulary of types (`ActionPlan`, `Message`, `ExecutionResult`, etc.). Building those first means Module 2 onward can be reviewed against a stable contract instead of a moving target.

Dependencies

Runtime: none (Python 3.11+ standard library only).
Dev/test: `pytest`, `pytest-asyncio`, `pytest-cov`, `mypy`, `ruff` (all optional — the test suite as shipped runs on `unittest` alone, zero installs required).

How to run

```
python -m unittest discover -s tests
-v
```

or, with dev tooling installed:

```
pip install -r requirements.txt
pytest -v
mypy nexus/
ruff check nexus/
```

Module 2: Security & Guardrail Layer ✅

Files added

```
nexus/security/__init__.py
nexus/security/validation.py      —
Structural plan validation
nexus/security/permissions.py     —
Permission profiles
(RESTRICTED/STANDARD/ELEVATED)
nexus/security/rate_limiter.py    —
Sliding-window rate limiter
nexus/security/confirmation.py    —
```

```
Single-use, replay-safe confirmation
tokens
nexus/security/audit.py            —
Append-only, JSON-lines audit trail
nexus/security/encryption.py       —
At-rest encryption (Fernet + no-op
fallback)
nexus/security/guardrail.py        —
DeterministicGuardrail engine
(orchestrates all above)
tests/test_security_validation.py
tests/test_security_permissions.py
tests/test_security_rate_limiter.py
tests/test_security_confirmation.py
tests/test_security_audit.py
tests/test_security_encryption.py
tests/test_security_guardrail.py
MODULE_2_SECURITY.md
```

Verified: `python -m unittest discover -s tests -v` → 105/105 passing (34 from Module 1 + 71 new).
All passing with zero warnings and an end-to-end smoke test.

Test coverage highlights:

- Validation: unknown action types, oversized fields, control char injection, scalar vs

complex param types

- Permissions: three default profiles, runtime switching, profile enforcement order
- Rate limiting: sliding window behavior, action expiry, concurrency
- Confirmation: single-use tokens, plan content hashing (reasoning field excluded), TTL expiry
- Audit: JSON format, concurrent writes, decision logging
- Encryption: Fernet roundtrip, tamper detection, key validation, production enforcement
- Guardrail: all five evaluation stages, precedence order, exception-based API, permission profile integration

Architecture

Five-stage evaluation pipeline (deterministic, fully testable, no randomness):

1. **Structural validation** — unknown action type or malformed plan → BLOCKED

2. **Hard denylist** — target matches restricted-app substring → BLOCKED (never overridable)
3. **Permission profile** — action type disallowed by current profile → BLOCKED
4. **Rate limiting** — too many actions in rolling minute → BLOCKED
5. **Confirmation required** — action type in `confirm_required_actions` → REQUIRES_CONFIRMATION (with single-use token)
6. Otherwise → ALLOWED

Every decision is audited to a dedicated JSON-lines file.

Key Design Decisions

- **Deterministic code, never delegated to an LLM.** A model asked "don't touch banking apps" can be tricked by adversarial input; a hard-coded denylist in the execution path cannot.
- **Fail closed.** Unknown action types, malformed plans, rate limits, etc. are BLOCKED, never silently allowed.

- **Confirmation is single-use and replay-safe.** A token is bound to the exact content hash of ONE plan; if the plan is mutated between user approval and execution, redemption fails.
- **Separate audit trail.** Different from general app logs; append-only, intended for real log-aggregation infrastructure in production.
- **One permission profile at a time.** NEXUS is single-user; you switch profiles (e.g., guest mode) via a single call.

Dependencies

Runtime (security): `cryptography` (optional, Fernet for at-rest encryption).
Module 1 has zero deps; Module 2 adds one optional dependency, kept optional so non-production deployments can skip it.

Next Steps

Module 3 candidates:

- **LLM Brain** (`nexus.agents`) — implements the Planner/Conversation/Vision agents,

with provider adapters for Claude/GPT/Gemini/Ollama.

- **Memory Layer** (`nexus.memory`) — long-term/short-term retrieval, vector stores, RAG.

I'd recommend **LLM Brain** next (everything downstream needs a working planner), but either direction works.